

InterpLens

An interactive mechanistic interpretability toolkit and visual debugger for LLM internals.

"Think VS Code debugger, but for transformer internals."

Built on top of [TransformerLens](#) and PyTorch, **InterpLens** gives researchers, students, and engineers an interactive GUI to inspect, visualize, and run causal experiments on decoder-only LLMs — replacing custom notebook code with a real-time visual workbench.

Key Features

-  **Logit Lens:** Track token prediction evolution layer by layer.
-  **Residual Stream Inspector:** Heatmaps & 2D PCA trajectories of hidden states.
-  **Attention Head Explorer:** $N \times N$ attention matrices & arc diagram visualizations.
-  **Neuron Activation Panel:** Rank top-firing MLP neurons and highlight text activation strips.
-  **Causal Patching & Tracing:** Swap activations between clean and corrupted prompts to discover causal circuits (ROME-style sweeps).
-  **SAE Feature Explorer:** Decompose hidden layers into sparse, monosemantic features.
-  **Model Comparison & Diff:** Compare internal representations of two models side-by-side.

Architecture & Documentation

- [System Architecture](#) — 4-tier model adapter & engine architecture.
- [Package Specification](#) — Class hierarchy, REST endpoints, and module design.
- [UI & Visual Specification](#) — Visual design tokens, component layout, and chart specs.
- [Development Roadmap](#) — Phased build plan from Phase 0 to Phase 7.
- [Original Design Plan](#) — Core vision and scope discipline.

Quickstart

1. Attaching to an Existing Model already on your GPU (Researcher Workflow)

If you already have a model loaded in GPU memory (e.g. inside a Jupyter notebook or research script), pass the model instance directly to `interplens.launch()` for zero-copy, zero-reload interpretability:

```
import interplens as il
from transformer_lens import HookedTransformer

# Suppose your model is already loaded on CUDA:
model = HookedTransformer.from_pretrained("gpt2-small", device="cuda")

# Attach InterLens debugger web UI without reloading or duplicating VRAM!
il.launch(model=model, port=8501)
```

2. Standard CLI Launch

```
pip install interplens

# Launch visual debugger GUI for any supported model
interplens launch --model gpt2 --device cuda
```

License

MIT License.